CARVALUE AI

Taller autoguiado de integración | Proyecto Big Data I

Duración: 150 minutos · **Equipos:** 2 o 3 · **Entrega:** repositorio GitHub con PRs revisados

Misión. Construid una aplicación web que estime el precio de un coche usado. El usuario introduce marca, año, kilometraje, combustible, carrocería y transmisión, y adjunta una foto. **Cohere Vision** determina el estado exterior de entre cuatro categorías; un **modelo de regresión ya entrenado** utiliza esa categoría junto al formulario y devuelve un precio. La tasación se guarda en **PostgreSQL** y queda disponible en un historial. Toda la infraestructura funciona en **Docker Compose**; se utiliza **Jupyter** para cargar los 50.000 coches y **Git/GitHub** para integrar el trabajo mediante ramas y Pull Requests.

> **No se utiliza JavaScript.** Todo se resuelve con HTML, formularios `POST` y plantillas Jinja2 en Flask. El objetivo es conectar servicios, no entrenar el modelo ni alcanzar métricas predictivas excelentes.

Objetivos verificables

Al terminar, otra persona podrá clonar vuestro repositorio y comprobar que:

1. `docker compose up --build -d` arranca **db, app y jupyter**; `http://localhost:5000/health` devuelve `status: ok`.
2. En Jupyter, el CSV **completo de 50.000 filas y 14 columnas** se importa a PostgreSQL en la tabla `anuncios` y una consulta `SELECT COUNT(*)` devuelve **50000**.
3. El formulario HTML recibe los seis datos del coche y una imagen JPG/PNG/WEBP sin JavaScript.

4. Cohere devuelve un estado exterior `fair`, `good`, `excellent` o `like new`, o `unknown` si la imagen es insuficiente. `unknown` **no debe inventarse** ni convertirse en una predicción.
5. El pipeline `.joblib` devuelve un precio estimado en dólares; se muestran **precio, estado, justificación y fuente**.
6. La tabla `tasaciones` guarda el resultado y la página `/historial` muestra las últimas 10.
7. **Cada integrante** ha creado su propia rama, realizado al menos **2 commits significativos** y abierto **al menos 1 PR**, revisado por un compañero y fusionado en `main`.

Qué debéis programar vosotros (importante)

Este material es un **esqueleto de proyecto**, no una solución con un par de espacios en blanco. Hay tres ejercicios de **autoría propia** que son especialmente importantes:

1. **Diseñar el prompt multimodal** en `app/vision.py`: recibir una imagen, obligar a elegir entre cuatro etiquetas, manejar `unknown` y pedir salida JSON. **No se proporciona ningún prompt terminado**, ni en el documento ni en el código.
2. **Construir desde cero el formulario HTML** en `templates/index.html`, y la vista de resultado en `templates/resultado.html`. Hay una plantilla base para los estilos, pero debéis escribir los campos, atributos y estructura.
3. **Orquestar la ruta POST de Flask** (`app/main.py`): recibir, validar, consultar visión, predecir, almacenar y responder. La ruta existe, pero su lógica central no está implementada.

Otras piezas se proporcionan hechas para no perder tiempo: modelo entrenado, validadores, parser de JSON, definición ORM, estilos CSS y listado del historial. Hay además TODOs pequeños en Docker, SQLAlchemy, Jupyter y predictor.

Material incluido: qué NO debéis rehacer

- `data/coches_50000_completo.csv`: CSV real preparado, 50.000 filas, **14 columnas**. ¡No lo reduzcáis ni generéis otros datos!
- `models/modelo_con_condition.joblib`: modelo de regresión **ya entrenado**, con el preprocesamiento incluido. **No se pide entrenarlo.**
- `app/validators.py`: validación de entradas y fotografías **completa**.
- `app/config.py`: valores válidos del CSV para los desplegables.

- `templates/base.html`, `templates/historial.html`: estilos globales y listado ya realizados. `index.html` y `resultado.html` **no vienen resueltos**.
- `tests/`: autocomprobaciones del núcleo que deben superar una vez integradas las ramas.
- `notebooks/01_importar_y_consultar.ipynb`: notebook con tres pequeñas tareas.

Arquitectura y contratos

```
Navegador --form POST /tasar--> Flask (app)
       Flask --foto--> Cohere Vision --> condition + reason
       Flask --seis datos + condition--> modelo.joblib --> precio
USD
       Flask --registro SQLAlchemy--> PostgreSQL (db): tasaciones
Jupyter (jupyter) --SQLAlchemy + Pandas--> PostgreSQL (db):
anuncios
       GET /historial <-- consulta tasaciones
```

El nombre de servidor para conectarse **entre contenedores es** `db`, no `localhost`. Las imágenes que sube el usuario **no se guardan** en la base de datos: solamente se almacena el estado y su justificación.

Plan de tiempo (150 minutos)

Momento	Actividad	Quién lidera
0-15 min	Crear repositorio, añadir colaboradores y abrir ramas	Todos
15-35 min	Completar Dockerfiles y Compose; primer arranque	A
35-55 min	Notebook: importar CSV completo y consultar PostgreSQL	A
55-80 min	SQLAlchemy + carga del `.joblib`	A y B
80-110 min	Programar ruta Flask y formulario HTML desde el esqueleto	B y C
110-130 min	Redactar prompt propio, conectar Cohere y probar (mock primero)	C, apoyo de B
130-150 min	PRs, revisión cruzada, merges, tests y entrega	Todos

Bonus opcional FUERA del núcleo de 150 minutos: dashboard de mercado con gráficos Matplotlib desde PostgreSQL. Consultad `docs/BONUS_ESTADISTICAS.md` cuando el proyecto principal ya funcione. No sacrifiquéis los PRs ni la prueba integrada por el bonus.

No esperéis al minuto 130 para subir código. Haced commits y PRs durante el trabajo. Los bloques pueden desarrollarse en paralelo, pero la prueba integral se hace tras integrar los PRs.

ETAPA 0 · Equipo, Git y PRs (0–15 min)

Requisito obligatorio: cada alumno desarrolla al menos una parte en su rama y abre un PR. Una persona que solo observe o toque el código de otro no cumple el requisito.

Reparto con 3 personas:

Rol	Rama propia	Archivos que toca
A - Infraestructura y datos	`feature/infra-datos`	`Dockerfile`, `Dockerfile.jupyter`, `compose.yaml`, `app/db.py`, `notebooks/*.ipynb`
B - API y predicción	`feature/api-ml`	`app/predictor.py`, `app/main.py`
C - Visión y web	`feature/vision-web`	`app/vision.py`, `templates/index.html`, `templates/resultado.html`

Con 2 personas: persona 1 hace **A + HTML** (comparte la tarea C de las plantillas); persona 2 hace **B + Cohere**. Dos ramas y dos PRs como mínimo. Evitad modificar el mismo archivo simultáneamente.

Creación del repositorio (solo una persona): crea en GitHub un repositorio **privado y vacío** (`carvalue-ai-grupo-X`) y añade a los demás como colaboradores con permiso de escritura. Desde la carpeta `01_ALUMNADO` ejecuta:

```
git init -b main
git add .
git commit -m "Base del taller CarValue AI"
git remote add origin https://github.com/USUARIO/carvalue-ai-
        grupo-X.git
git push -u origin main
```

Cada compañero clona el repo y crea su propia rama:

```
git clone https://github.com/USUARIO/carvalue-ai-grupo-X.git
cd carvalue-ai-grupo-X
git switch -c feature/api-ml     # ejemplo: cambia por tu rama
git status
```

Cada cual trabaja **solo en los archivos de su responsabilidad**. Recordatorio:

```
git add app/predictor.py
git commit -m "Integrar modelo joblib en predictor"
git push -u origin feature/api-ml
```

Luego, desde GitHub: **Compare & pull request → base: main → revisar diferencias → crear PR**. Una persona distinta revisa los cambios, deja un comentario concreto y aprueba el PR si los permisos lo permiten. Después se fusiona desde GitHub. **No hagáis commits directos en** `main`, salvo el commit inicial del repositorio.

> Si GitHub no permite que un colaborador apruebe formalmente el PR por la configuración del repositorio, su comentario de revisión debe dejar constancia explícita de la revisión. Un PR sin revisión cruzada no satisface la práctica.

Comprobación de la etapa: todos tienen acceso al repositorio, su rama propia y un primer commit o fichero identificado para trabajar.

ETAPA 1 · Docker / Compose (15–35 min) · ROL A

En local (Windows con Docker Desktop y WSL2 habilitado, macOS o Linux): ejecutad `docker --version` y `docker compose version`. Copiad `.env.example` a `.env` (en Windows PowerShell: `Copy-Item .env.example .env`; en Git Bash/Linux: `cp .env.example .env`). **No subáis** `.env` **a Git**.

En `compose.yaml` tenéis tres servicios (`db`, `app`, `jupyter`) y una red interna de Compose ya preparada. Completad:

- **A1:** variables de entorno de `app`: `COHERE_API_KEY: ${COHERE_API_KEY:-}` y `VISION_MOCK: ${VISION_MOCK:-1}`.
- **A2:** variable de `jupyter`: `JUPYTER_TOKEN: ${JUPYTER_TOKEN:-carvalue123}`.
- **A3:** en `Dockerfile` activar la instalación de `requirements.txt` mediante `pip install --no-cache-dir -r requirements.txt`.
- **A4:** en `Dockerfile.jupyter` instalar `requirements_jupyter.txt` de forma equivalente.

Comprobad la sintaxis y arrancad:

```
docker compose config
docker compose up --build -d
docker compose ps
docker compose logs --tail=40 app
```

Comportamiento esperado: tres servicios activos. La base de datos pasa a `healthy`, Flask escucha en 5000 y Jupyter en 8888. Visitad `http://localhost:5000/health`, y `http://localhost:8888/lab?token=carvalue123` (cambiad el token si lo habéis modificado). El formulario inicial puede verse antes de implementar la predicción.

Atención: `db` es el host de PostgreSQL desde Flask/Jupyter. `localhost` **dentro de un contenedor apunta a ese mismo contenedor**, no a PostgreSQL.

Comprobación de la etapa: `docker compose ps` muestra servicios listos y la URL `/health` responde.

ETAPA 2 · Jupyter y PostgreSQL (35–55 min) · ROL A

En Jupyter abrid `notebooks/01_importar_y_consultar.ipynb`. El notebook ya tiene las importaciones, la URL de conexión a PostgreSQL en `DATABASE_URL` y la ruta al CSV.

- **A7:** rellenad `df.to_sql('anuncios', ...)` con `if_exists='replace'`, `index=False`, `chunksize=1000` y, opcionalmente, `method='multi'`.
- **A8:** sustituid `SELECT 1 AS total` por `SELECT COUNT(*) AS total FROM anuncios`.
- Ejecutad la consulta de agrupación por `condition` que ya está implementada.

Comprobación real: la primera consulta tiene que mostrar **50000**. No vale un `print(len(df))` como prueba de que los datos están en la base de datos: exigimos la consulta SQL.

```sql
SELECT COUNT(*) AS total FROM anuncios;
SELECT condition, COUNT(*) FROM anuncios GROUP BY condition;
```

Las 14 columnas se importan porque esta fase trata de **ingesta**; el modelo solo necesita siete de ellas. Conservad el CSV completo.

ETAPA 3 · Modelo y persistencia (55–80 min) · ROLES A y B

A: `app/db.py`. El motor SQLAlchemy, el modelo ORM `Tasacion` y `crear_tablas()` ya están dados.

- **A5** `guardar_tasacion(registro)`: abrir `Session(engine)`, construir `Tasacion(**registro)`, `session.add`, `session.commit`, `session.refresh` y devolver el identificador.
- **A6** `obtener_ultimas_tasaciones(limite)`: hacer `select(Tasacion).order_by(Tasacion.id.desc()).limit(limite)` y retornar los registros mediante `session.scalars(...)`.

La tabla `tasaciones` se crea automáticamente al arrancar Flask, de modo que no debéis escribir un `CREATE TABLE` a mano.

B: `app/predictor.py`. Ya están `MODELO_RUTA`, `COLUMNAS_MODELO` y `cargar_modelo()`; el fichero `.joblib` incluye preprocesamiento y regresor.

- **B1** `predecir_precio(datos, estado)`: unir `datos` con `condition`, crear `pd.DataFrame([registro], columns=COLUMNAS_MODELO)`, invocar `.predict(...)` y devolver `float` (con redondeo razonable).

Contrato exacto de entrada al modelo: `year`, `odometer`, `manufacturer`, `fuel`, `type`, `transmission`, `condition`. Las millas, no kilómetros. Las categorías deben utilizarse con los **valores ingleses** del dataset.

Comprobación: una predicción numérica positiva para `toyota`, `2016`, `80000` millas, `gas`, `sedan`, `automatic`, `good`.

ETAPA 4 · Flask y HTML: implementación propia (80–110 min) · ROLES B y C

Aquí hay **dos bloques que vais a escribir vosotros casi desde cero**: el flujo de la ruta `POST /tasar` y el formulario web. El objetivo no es escribir código complicado, sino comprender cómo se conectan peticiones HTTP, validaciones, servicios, predicción y HTML renderizado por el servidor.

B · `app/main.py`: construir el flujo de negocio (TODO B2)

La ruta `POST /tasar` está declarada y ya contiene un bloque `try/except`. Dentro de él **debéis escribir la lógica restante**, siguiendo este orden:

1. Recuperar los campos de texto del formulario con `request.form` y la fotografía desde `request.files` usando exactamente el nombre `foto`. Ya están disponibles `validar_datos` y `validar_foto`, que devuelven respectivamente un diccionario normalizado y `(bytes, mimetype)`.
2. Llamar a `clasificar_estado(bytes, mimetype)`. Su contrato devuelve tres valores: `estado`, `razon`, `fuente`. `fuente` debe ser `MOCK` o `COHERE` según qué mecanismo se haya usado.
3. Si se recibe `unknown`, **detener el proceso** y mostrar un error comprensible: no se debe asignar arbitrariamente `good` ni guardar una tasación.
4. Construir la llamada a `predecir_precio(datos, estado)` y recuperar un `float` que expresa dólares.
5. Crear un diccionario con las seis variables originales y cuatro datos adicionales: `condition`, `justificacion`, `price_prediction`, `vision_source`. Guardarlo mediante `guardar_tasacion`, obteniendo su identificador.
6. Renderizar `resultado.html` pasando los nombres de variables exactos `datos`, `estado`, `razon`, `precio`, `fuente`, `identificador`.

Preguntas para comprobar comprensión: ¿por qué la imagen entra desde `request.files` y no desde `request.form`? ¿Por qué el modelo de ML no debe llamarse cuando Cohere devuelve `unknown`? ¿Dónde se devuelve la página HTML final al navegador?

Prueba de B: con modo `VISION_MOCK=1` y el formulario completo, tras un POST válido debe aparecer una página HTML de resultado y un registro nuevo en `/historial`. Si no, revisad el contrato en `docs/CONTRATO_COMPONENTES.md`.

C · `templates/index.html`: escribir el formulario completo (TODO C2)

Se facilita solo la estructura de la página y el bloque que muestra un mensaje de error. **Debéis construir el formulario y sus controles HTML vosotros mismos**. No se os da el formulario resuelto ni un fragmento listo para pegar.

Requisitos **obligatorios y verificables**:

Campo esperado por Flask	Control HTML que elegiréis	Restricción
`manufacturer`	Desplegable	Sus opciones proceden de `opciones.marcas`
`year`	Numérico	De 1995 a 2021; obligatorio
`odometer`	Numérico	De 1.000 a 250.000; representa **millas**
`fuel`	Desplegable	Sus opciones proceden de `opciones.combustibles`
`type`	Desplegable	Sus opciones proceden de `opciones.tipos`
`transmission`	Desplegable	Sus opciones proceden de `opciones.transmisiones`
`foto`	Archivo	JPG, PNG o WEBP; obligatorio

El formulario debe apuntar a la ruta `/tasar`, enviar por **POST**, utilizar la codificación adecuada para **archivos adjuntos** y tener un botón de envío. Cada control necesita el atributo `name` exacto de la tabla, una etiqueta legible y una restricción adecuada cuando corresponda. Para llenar los desplegables, **usad bucles Jinja2** con las listas ya recibidas por la plantilla: no copiéis manualmente todas las marcas y combustibles. El CSS ya está proporcionado en `base.html` (clases `grid`, `card`, `hint`, etc.); no se evalúa el diseño visual.

No debe existir un selector de `condition`: **es el valor que se obtiene de la foto mediante Cohere**.

Preguntas útiles para guiaros: ¿cuáles son los atributos HTML que necesita un archivo? ¿Qué diferencia hay entre `name` e `id`? ¿Cómo se utiliza una variable `opciones.marcas` en un bucle de Jinja2?

C · `templates/resultado.html`: presentación de la tasación (TODO C3)

Escribid el bloque HTML + Jinja2 que muestre **el número de tasación, la marca y año, el precio con separador de miles y símbolo de USD, la categoría devuelta, la razón proporcionada y la fuente** (`MOCK` o `COHERE`). Incluir un enlace que vuelva al formulario. Las variables están descritas en `docs/CONTRATO_COMPONENTES.md`.

Prueba de C: comprobar en el navegador que el usuario ve todos los campos, puede elegir un JPG/PNG/WEBP, que la ruta recibe el archivo y que tras tasar se ve una página real de resultados. No usar JavaScript ni `fetch()`.

Comprobación conjunta B+C

Con `VISION_MOCK=1`, enviar un coche de ejemplo (`toyota`, 2016, 80.000 millas, `gas`, `sedan`, `automatic`) y una foto válida. La fuente de clasificación debe mostrarse como **MOCK** y la categoría será `good`. Esta prueba **no acredita** haber analizado la foto con Cohere; únicamente verifica el circuito HTTP → ML → PostgreSQL → HTML. Contrastad el nuevo registro en `/historial`.

ETAPA 5 · Cohere Vision: redactad vuestro prompt (110–130 min) · ROL C

El módulo `app/vision.py` tiene disponible el modo de pruebas `MOCK`, un comprobador de JSON llamado `parsear_respuesta`, y la estructura de `clasificar_estado`. **El valor `PROMPT` está vacío intencionalmente**. Redactarlo forma parte del ejercicio: no hay una solución para copiar en el proyecto del alumnado.

C1-A · Escribir el prompt propio

Vuestro prompt debe especificar, de forma inequívoca, **todos** estos requisitos. Podéis elegir la redacción y el orden; lo que se comprueba es el comportamiento:

1. **Rol y tarea:** analizar la fotografía subida para evaluar **solo el aspecto EXTERIOR visible** del vehículo, y elegir un único estado.
2. **Vocabulario cerrado:** aceptar solamente los valores exactos `like new`, `excellent`, `good`, `fair`. Debéis describir el significado de cada uno: como nuevo, excelente con desgaste mínimo, bueno con marcas pequeñas y aceptable con daños o desgaste visibles.
3. **Foto insuficiente:** si no se aprecia claramente un coche o faltan evidencias visuales, devolver `unknown`. No obligar a adivinar.
4. **No inferir lo invisible:** queda prohibido concluir estado de motor, historial de accidentes, kilometraje real, averías o funcionamiento mecánico a partir de la imagen.
5. **Contrato de salida:** solicitar una respuesta interpretable como **JSON**, sin párrafos introductorios ni bloques Markdown, con las claves exactas `condition` y `reason`. La primera contiene una etiqueta inglesa admitida; la segunda, una explicación **breve en español** basada en detalles visibles.

6. **Robustez y precisión:** pedir una sola etiqueta y una justificación concisa; no generar campos adicionales ni puntuaciones numéricas de confianza inventadas.

Un ejemplo de salida que vuestro sistema debe ser capaz de procesar (esto NO es un prompt):

```
{"condition": "good", "reason": "Se aprecian marcas leves de uso
        en la carrocería."}
```

Cómo valorar el prompt: probad al menos una foto nítida de un coche y otra que no permita evaluarlo (por ejemplo, un recorte de una rueda). Revisad si devuelve una de las etiquetas permitidas o `unknown`, si evita afirmaciones sobre el motor y si la respuesta pasa `parsear_respuesta`. Documentad en `ENTREGA_GRUPO.md` una decisión de diseño del prompt que hayáis tomado.

C1-B · Implementar la llamada multimodal

Usad la documentación oficial `https://docs.cohere.com/docs/image-inputs` (modelo `command-a-vision-07-2025`). **Implementad vosotros** la lógica que falta:

- El código ya lee `COHERE_API_KEY`. Si está vacía, debe avisar; nunca incrustéis la clave en el repositorio.
- Convertid los bytes de la foto a Base64 y preparad una URL de datos (`data:`) que respete el MIME de la imagen; la URL se envía a Cohere, no se guarda en PostgreSQL.
- Utilizad el cliente Python `cohere.ClientV2` con el método `chat`. En el mensaje de usuario deben viajar **dos elementos**: el texto que vosotros habéis escrito en `PROMPT` y la imagen en formato `image_url`.
- Extraed los textos del contenido devuelto por la API, pasadlos a `parsear_respuesta` (proporcionado) y devolved la terna `(estado, razon, "COHERE")`.
- Controlad los errores de red, autenticación y respuesta no válida usando `VisionError`. No devolváis `good` como solución por defecto a un fallo.

Importante: os damos los nombres y el contrato de la API, no una función completa ni un prompt. Leed documentación, programad y depurad vuestra integración.

Pruebas en dos etapas

1. Sin Cohere: `VISION_MOCK=1` en `.env` y reiniciad el contenedor `app` para que tome el cambio. Devuelve siempre `good` y fuente `MOCK`, sin analizar la foto ni consumir API. Con este modo verificamos el circuito completo.

2. Con Cohere: `VISION_MOCK=0`, clave válida en `.env` y reiniciar `app`. Subid un JPG/PNG/WEBP exterior real. Debe aparecer fuente `COHERE`, estado entre cuatro etiquetas o un mensaje comprensible si devuelve `unknown`. Si la API no está disponible o supera un límite, documentadlo con honestidad: **el mock nunca puede presentarse como la ejecución real del análisis visual**.

No almacenéis fotografías ni credenciales en Git. El valor `condition` obtenido por visión es una estimación subjetiva del estado exterior; no tiene que coincidir con una inspección física profesional.

ETAPA 6 · Pull Requests, tests y entrega (130–150 min)

Acordad contratos y fusionad en orden razonable: primero infraestructura/datos, después API/modelo y finalmente interfaz/visión. Para cada PR, un compañero **distinto al autor** revisa el diff y escribe un comentario que incluya algo concreto (por ejemplo: «Verificada la conexión con SQLAlchemy; falta/muy bien el manejo de errores»).

1. Cada integrante hace **mínimo 2 commits relevantes** en su rama y abre su PR.
2. Todos los PRs tienen revisión cruzada y quedan **merged** en `main`.
3. Cread un clon limpio o actualizad `main`, volved a levantar el proyecto y ejecutad:

```
git switch main
git pull --ff-only
docker compose up --build -d
docker compose exec app pytest -q
```

1. Verificad manualmente `http://localhost:5000`, `http://localhost:5000/historial`, `http://localhost:5000/api/tasaciones`, `http://localhost:5000/health`, y `SELECT COUNT(*) FROM anuncios` en Jupyter.

2. Cambiad temporalmente a `VISION_MOCK=0` y demostrad una tasación real con Cohere si tenéis clave disponible; volved a modo mock si necesitáis que los tests no consuman API.
3. Rellenad `ENTREGA_GRUPO.md` con nombres, papeles, **enlace al repositorio**, **enlace a cada PR**, estado de prueba con Cohere y qué os ha resultado más difícil.

Comprobación final de aceptación (marcad en `ENTREGA_GRUPO.md`)

- [ ] Tres servicios Docker Compose activos.
- [ ] CSV de 50.000 filas importado a `anuncios`, comprobado con SQL.
- [ ] Modelo `.joblib` predice con cuatro estados aceptados.
- [ ] Formulario HTML sin JavaScript y con foto JPG/PNG/WEBP.
- [ ] App en modo mock funcionando y resultado guardado en `tasaciones`.
- [ ] Modo real Cohere probado o incidencia documentada sin falsear la prueba.
- [ ] `/historial` enseña la nueva tasación.
- [ ] Cada alumno aportó commits, PR y revisión cruzada.
- [ ] `.env` no está en Git.

Entrega: únicamente el enlace al repositorio privado con acceso de lectura al profesor y sus PRs accesibles. No es necesario entregar capturas si se puede verificar el repositorio y el funcionamiento.

Solución de problemas rápida

Síntoma	Causa probable y qué comprobar
`ModuleNotFoundError: flask/sqlalchemy/...`	Falta habilitar `RUN pip install` en alguno de los Dockerfiles. Ejecutar `docker compose up --build -d`.
`Connection refused` / `could not translate host name`	En los contenedores, PostgreSQL se llama `db` y escucha en 5432. Ver `DATABASE_URL` y `docker compose ps`.
`relation "anuncios" does not exist`	El notebook no ha ejecutado `df.to_sql` o se está consultando otra base de datos.

Síntoma	Causa probable y qué comprobar
Jupyter pide token	Revisar `JUPYTER_TOKEN` en Compose y `.env`. Usar la URL con `?token=...`.
`request.files` vacío	Falta `enctype="multipart/form-data"` o el campo `name="foto"`.
Error con `joblib` / sklearn	El modelo se guardó con scikit-learn 1.8.0; mantener esa versión. Revisar las siete columnas y tipos.
Modo mock siempre devuelve `good`	Es correcto: **NO** analiza la imagen. Para Cohere real hay que poner `VISION_MOCK=0` y reiniciar el contenedor.
Cohere devuelve otra categoría o `unknown`	No forzar la tasación. Probar imagen clara del exterior; revisar salida del proveedor y prompt.
`401` / `429` de Cohere	Clave inválida o límite de uso. Documentar incidencia. Nunca compartir el token en Git.
El puerto 5000 o 8888 está ocupado	Cerrar otro servicio o ajustar **solo el puerto del host** en Compose.
Cambios de Python no se ven	Los archivos `app/` y `templates/` están montados como volúmenes; revisar `docker compose logs -f app`. Si hay cambios en dependencias o modelos, reconstruir.
Conflicto entre ramas	No editar el mismo fichero a la vez. Actualizar `main` y resolver el diff con el equipo; documentar el conflicto en el PR.

Bonus voluntario · Analítica visual de PostgreSQL (fuera de los 150 minutos del núcleo)

Caso de uso: un compraventa quiere conocer el **precio mediano** y el **volumen de anuncios** por categoría de estado, para contextualizar el precio estimado por CarValue AI. El dashboard debe utilizar datos **leídos de PostgreSQL**, no directamente del CSV.

- Implementar las tareas D1-D4 descritas paso a paso en `docs/BONUS_ESTADISTICAS.md`.
- Crear `GET /estadisticas` (página HTML) y `GET /estadisticas/grafico.png` (imagen de Matplotlib renderizada por Flask). No usar JavaScript.
- Producir al menos un gráfico de barras: precio mediano por categoría `condition` de la tabla `anuncios`; como segundo gráfico opcional, cantidad de anuncios por estado.
- Si no se ha cargado el CSV desde Jupyter, explicar que faltan datos. Evitad cifras incrustadas a mano.
- Desarrollar el bonus en la rama `feature/dashboard` cuando el núcleo ya esté integrado y abrir **otro PR revisado**.

El bonus no sustituye ninguna tarea obligatoria ni los PRs personales de los integrantes. Es una extensión de negocio para quienes terminen antes o quieran completarla después.

Preguntas de reflexión (2 minutos, en el fichero de entrega)

1. ¿Qué se comunica usando HTTP y qué se comunica usando una conexión SQL?

2. ¿Por qué `db` es el host correcto desde Flask, mientras que `localhost:5000` se usa desde el navegador?
3. ¿Qué ventaja aporta un PR frente a subir todos los cambios directamente a `main`?
4. ¿Qué sucede si Cohere responde `unknown`? ¿Por qué no debemos inventar un valor?

Fuentes de referencia

- Cohere Vision: https://docs.cohere.com/docs/command-a-vision
- Cohere imágenes: https://docs.cohere.com/docs/image-inputs
- Docker Compose: https://docs.docker.com/compose/
- Flask: https://flask.palletsprojects.com/
- SQLAlchemy: https://docs.sqlalchemy.org/
- Dataset origen: https://www.kaggle.com/datasets/austinreese/craigslist-carstrucks-data